

THE UNIVERSITY OF BRITISH COLUMBIA



Electrical and Computer Engineering

CPEN 502: Architecture for Learning Systems

ASSIGNMENT #1 – Back Propagation

Student Name: Navraj Singh Kambo	Student Number: 23152151	Set: _

Table of Contents

1 Introduction 4
2 Back Propagation Learning 4
2.1 Part 1a 5
2.2 Part 1b 7
2.3 Part 1c 9
2.4 Part 1d 10
2.4.1 a 10
2.4.2 b 11
2.4.3 c 12
3 Appendix 13
3.1 NNInterface 13
3.2 BaseInterface 16
3.3 LUTInterface and Training Sets 22

Table of Figures

Figure 1 – Error vs Epochs over 100 trials – Binary (Part 1a) 5
Figure 2 – Error vs Epochs – single trial (Part 1a) 5
Figure 3 – Net plot (Part 1a) 6
Figure 4 – Error vs Epochs over 100 trials – Bipolar (Part 1b) 7
Figure 5 – Error vs Epochs – single trial (Part 1b) 7
Figure 6 – Net plot (Part 1b) 8
Figure 7 – Error vs Epochs over 100 trials – Bipolar w/ Momentum (Part 1c) 9
Figure 8 – Error vs Epochs – single trial (Part 1c) 9
Figure 9 – Error vs Epochs over 100 trials – Binary (Part 1d)(a) 10
Figure 10 – Error vs Epochs – single trial (Part 1d)(a) 10
Figure 11 – Error vs Epochs over 100 trials – Bipolar (Part 1d)(b) 11
Figure 12 – Error vs Epochs – single trial (Part 1d)(b) 11
Figure 13 – Error vs Epochs over 100 trials – Bipolar w/ Momentum (Part 1d)(c) 12
Figure 14 – Error vs Epochs – single trial (Part 1d)(c) 12
Figure 15 – NNInterface.py 16
Figure 16 – BaseInterface.py 21
Figure 17 – LUTInterface.py 22

Figure 18 – main.py 22
Figure 19 – BinaryTrainingSet.yaml 22
Figure 20 – BipolarTrainingSet.yaml 22

Table of Tables

No table of figures entries found.

1 Introduction

For this part of the assignment you are to implement a multi-layer perceptron and train it using the error-backpropagation algorithm. You will not yet need to use Robocode, however you should keep in mind that your neural net will later be used in your robot tank. It will be useful if your neural network software is able to accept as a parameter the number of hidden neurons. (Later on you may wish to also add support for multiple layers).

2 Back Propagation Learning

Set up your network in a 2-input, 4-hidden and 1-output configuration. Apply the XOR training set. Initialize weights to random values in the range -0.5 to +0.5 and set the learning rate to 0.2 with momentum at 0.0.

2.1 Part 1a

Define your XOR problem using a binary representation. Draw a graph of total error against number of epochs. On average, how many epochs does it take to reach a total error of less than 0.05? You should perform many trials to get your results, although you don't need to plot them all.

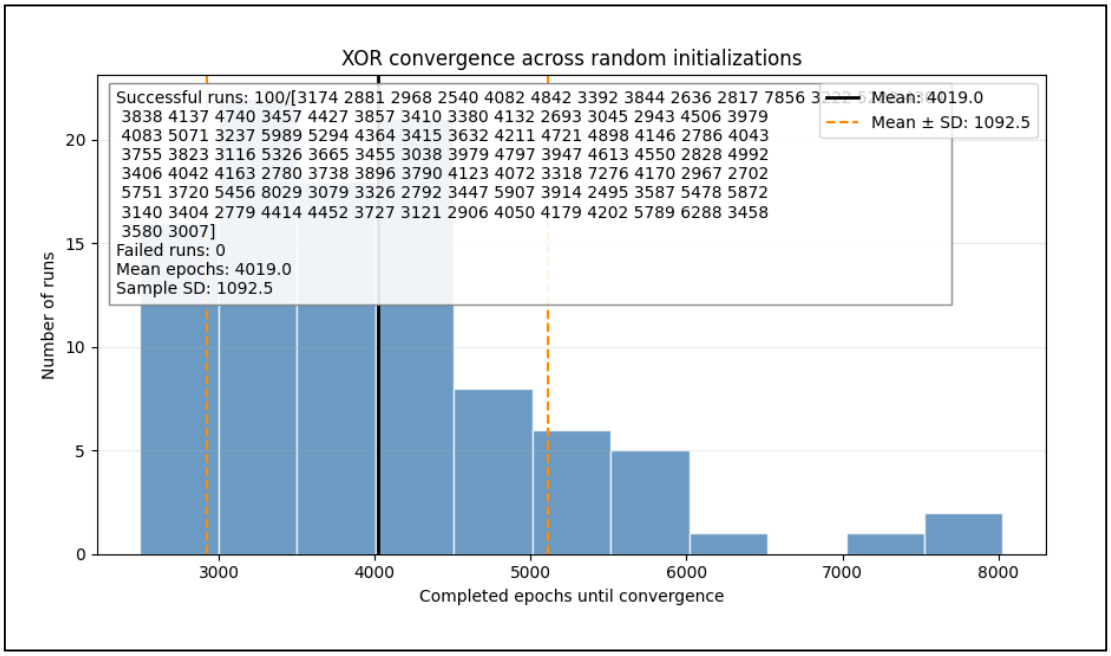


Figure 1 – Error vs Epochs over 100 trials – Binary (Part 1a)

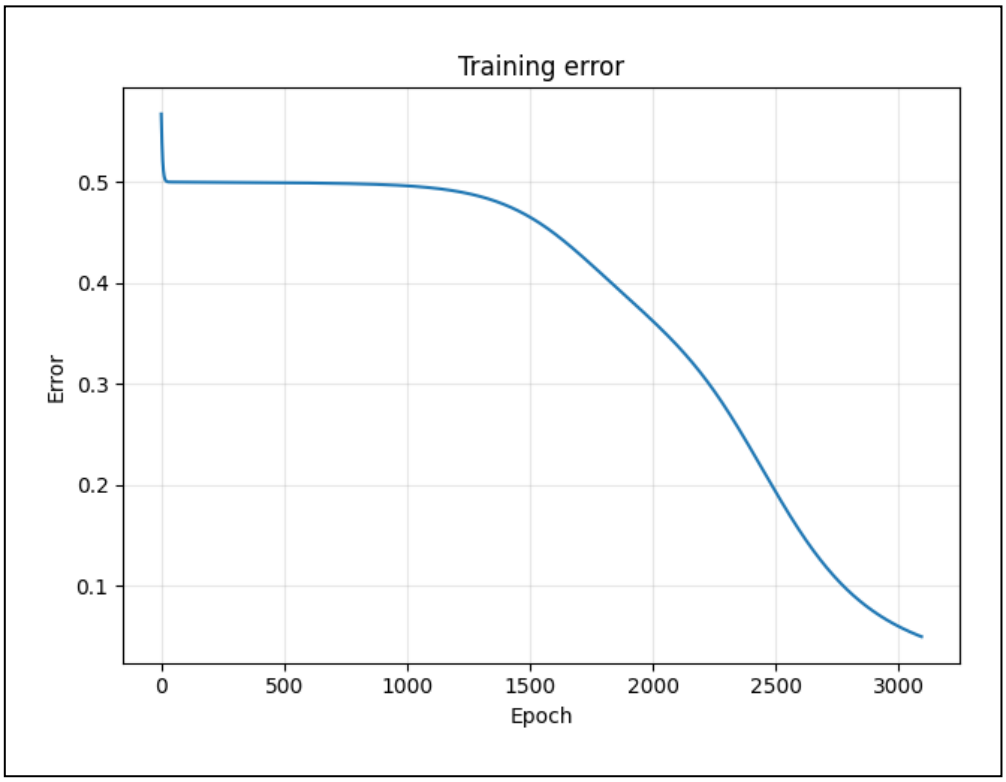


Figure 2 – Error vs Epochs – single trial (Part 1a)

On average, it seemed to reach a total error of less than 0.05 around ~3000-4000 epochs.

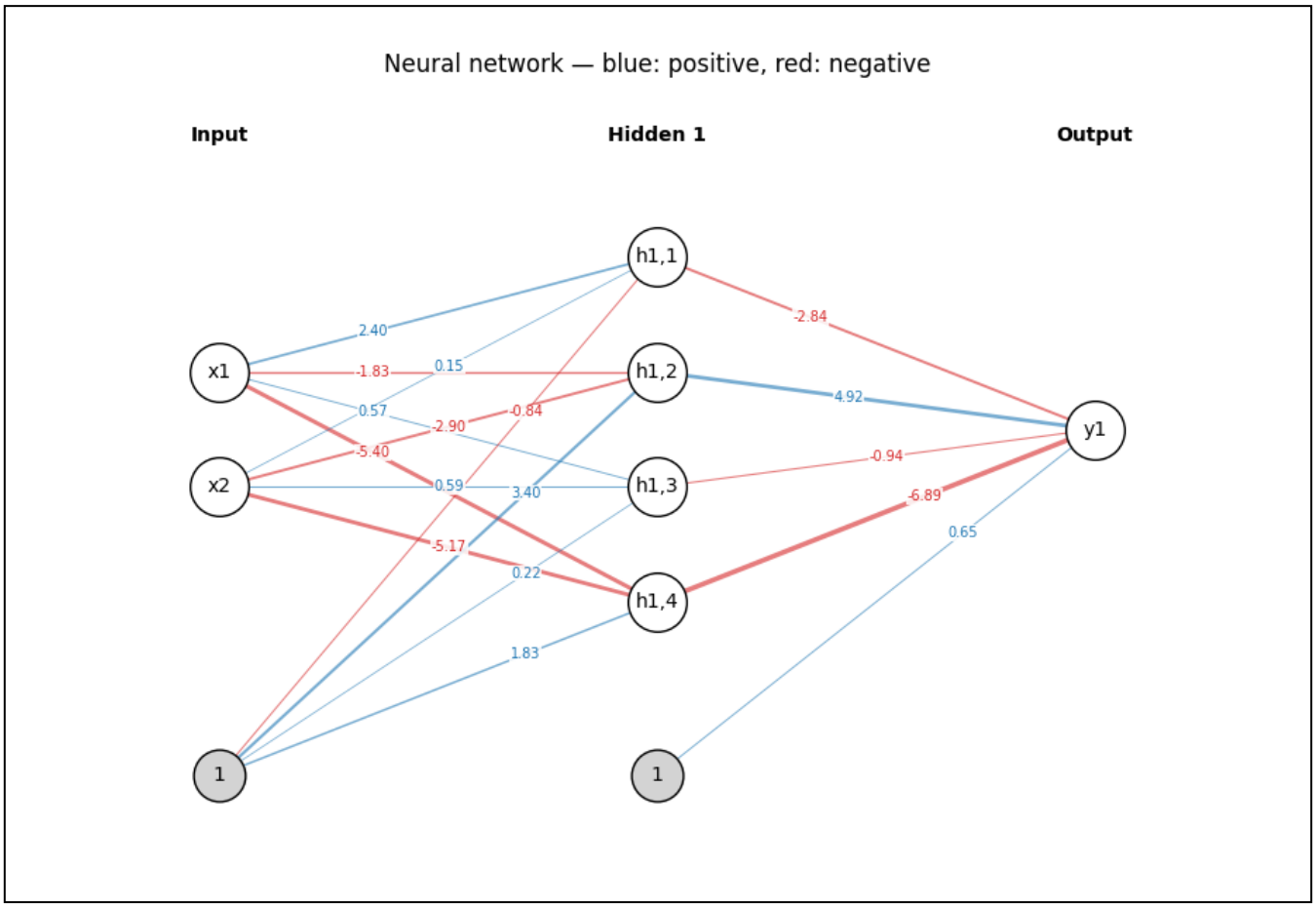


Figure 3 – Net plot (Part 1a)

2.2 Part 1b

This time use a bipolar representation. Again, graph your results to show the total error varying against number of epochs. On average, how many epochs to reach a total error of less than 0.05?

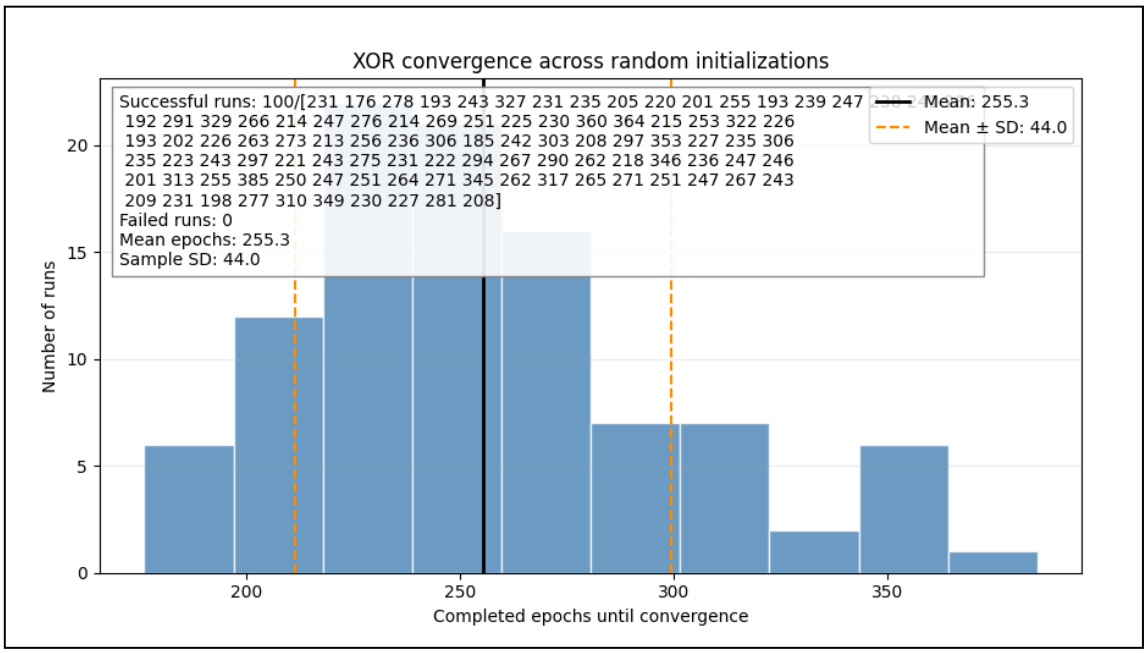


Figure 4 – Error vs Epochs over 100 trials – Bipolar (Part 1b)

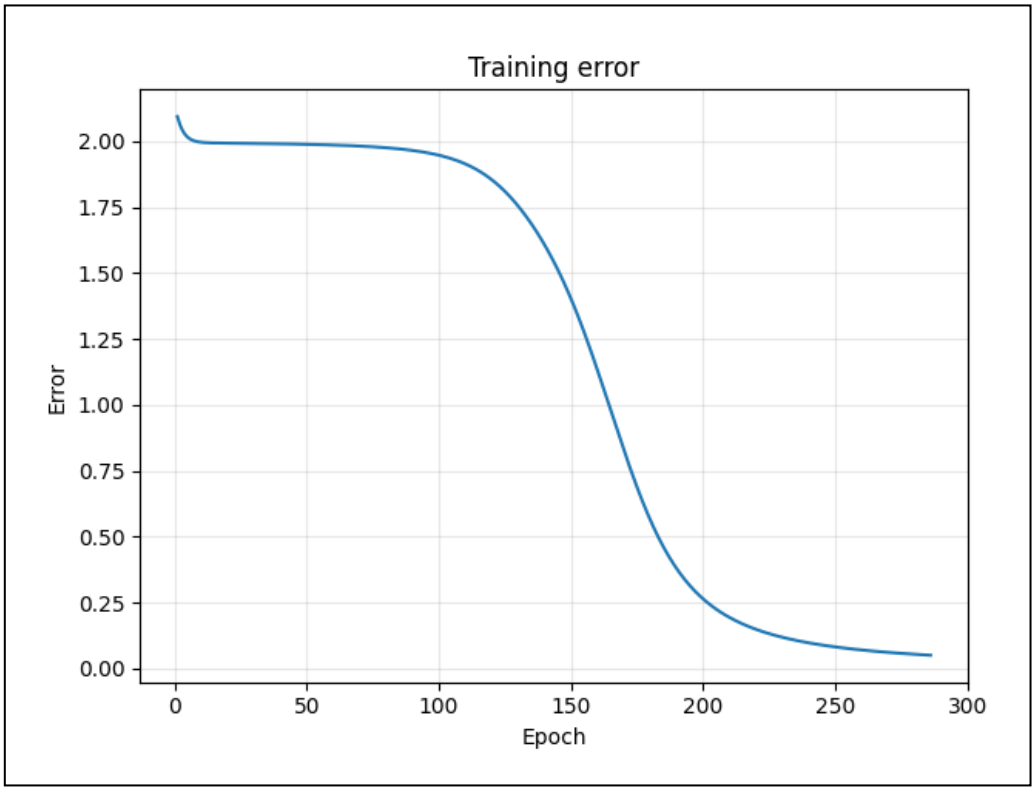


Figure 5 – Error vs Epochs – single trial (Part 1b)

On average, it seemed to reach a total error of less than 0.05 around ~300 epochs.

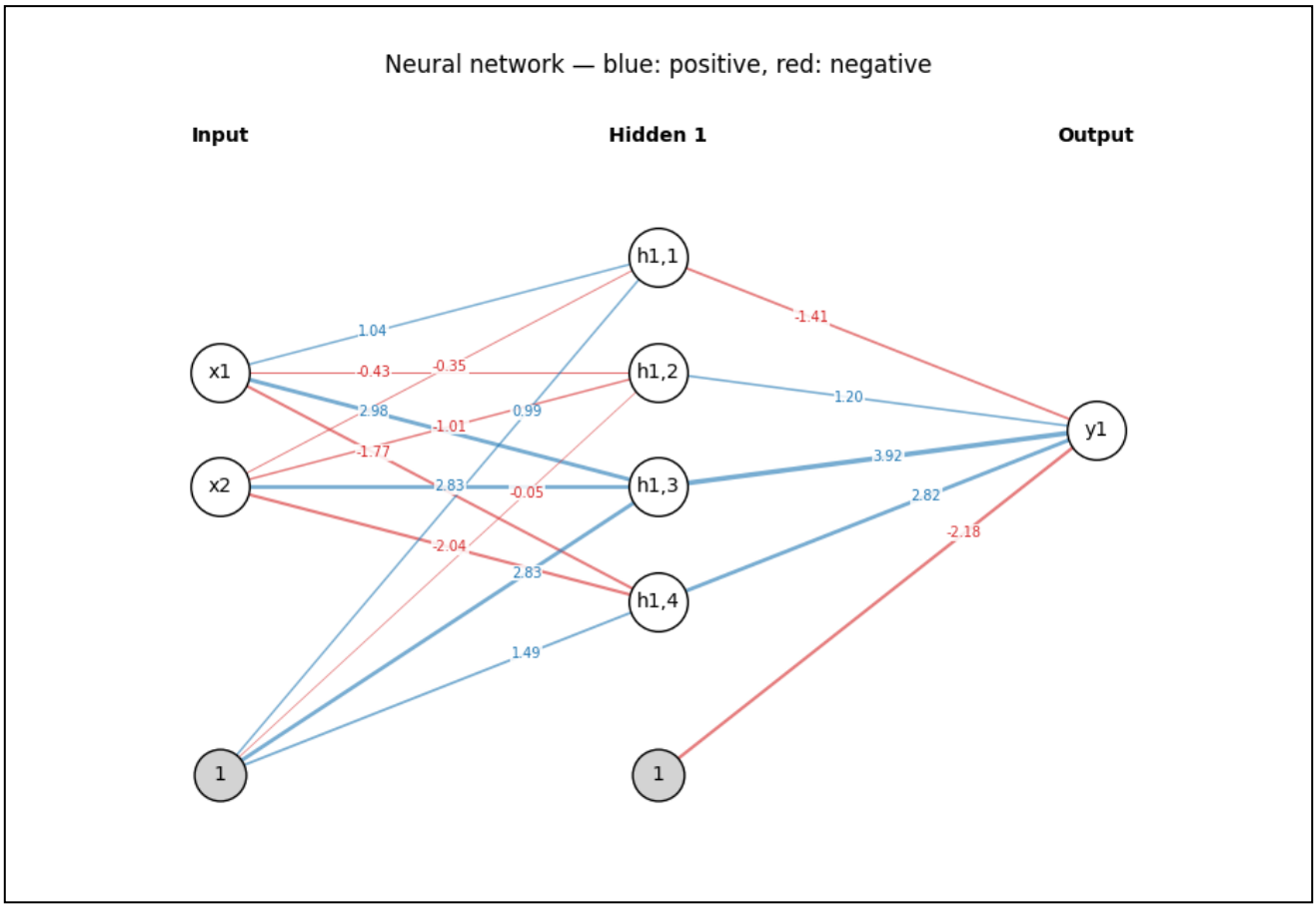


Figure 6 – Net plot (Part 1b)

2.3 Part 1c

Now set the momentum to 0.9. What does the graph look like now and how fast can 0.05 be reached?

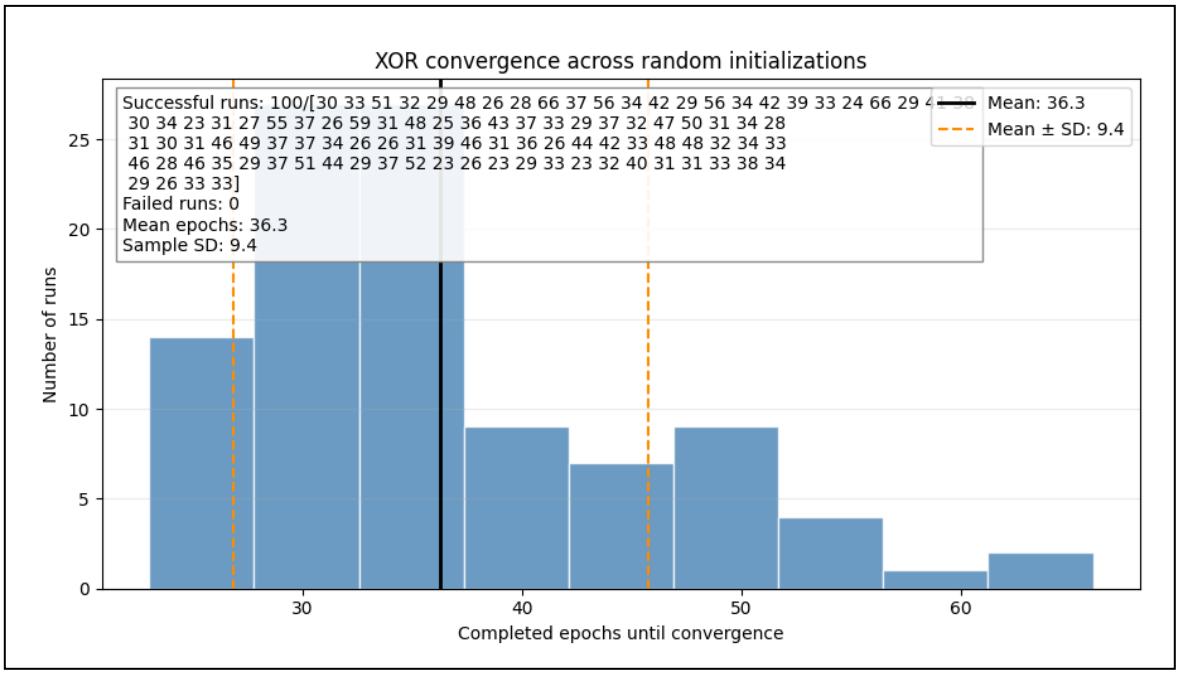


Figure 7 – Error vs Epochs over 100 trials – Bipolar w/ Momentum (Part 1c)

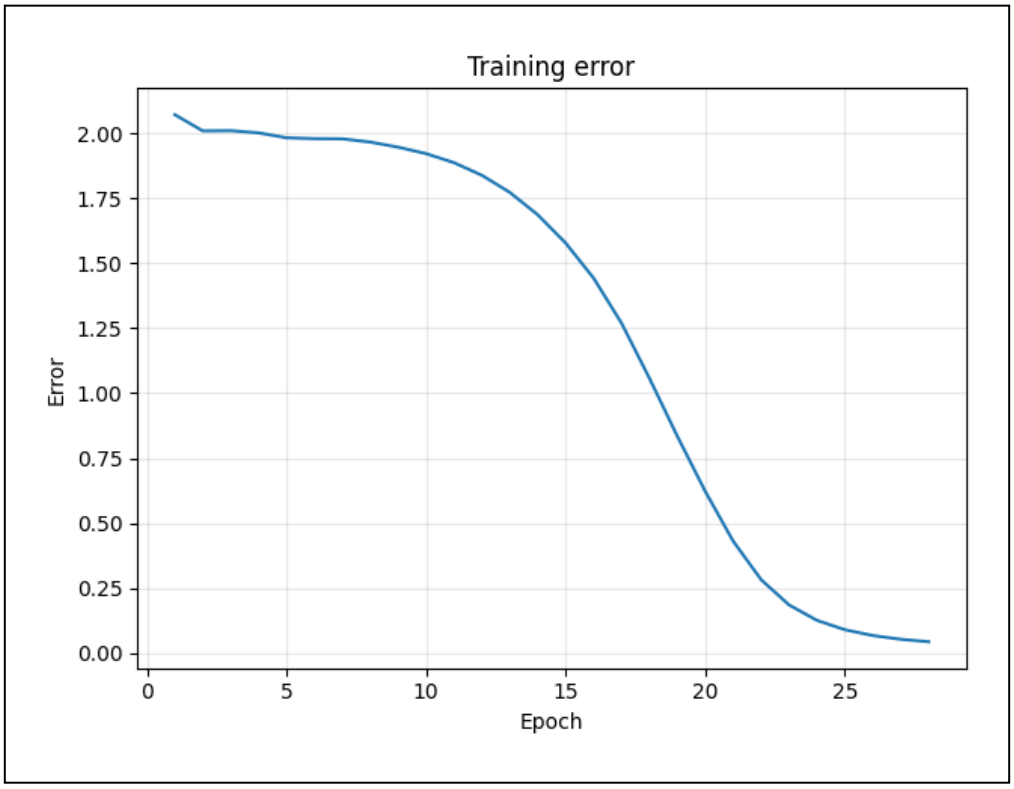


Figure 8 – Error vs Epochs – single trial (Part 1c)

The plot still looks like the same S-shaped curve; however, it is steeper. It seemed to reach a total error of less than 0.05 around ~30 epochs (on-average) and had a significantly smaller standard deviation.

2.4 Part 1d

In modern deep neural networks, training is often performed in mini batches. Where a mini batch is a sample of data points from the full training data set. Gradients are computed in parallel on a GPU for the sample and an aggregated weight change computed for the batch and applied once. Implement batch gradient descent (CPU ok 😊) and compare training behaviour to your current online, one-by-one weight update variant. Plot and show graphs for each of the 3 scenarios above. Comment on any differences you observe in training behaviour.

2.4.1 a

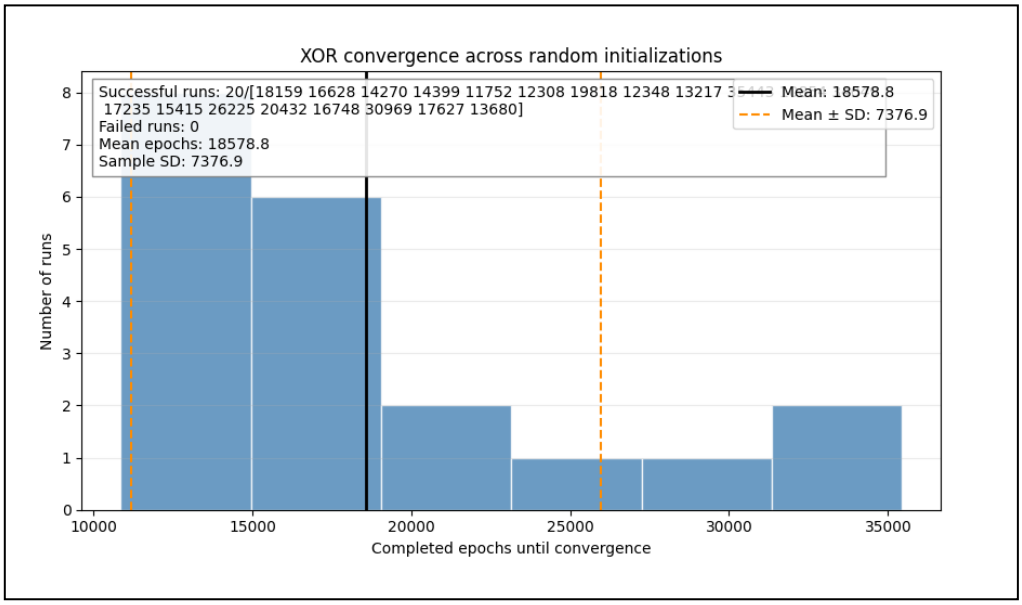


Figure 9 – Error vs Epochs over 100 trials – Binary (Part 1d)(a)

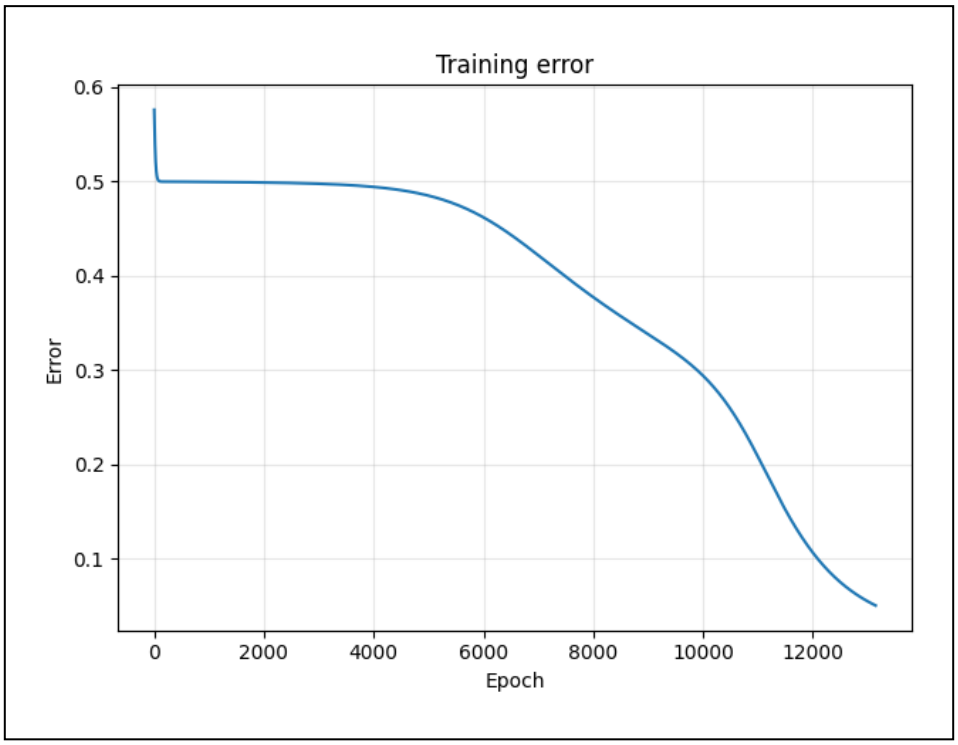


Figure 10 – Error vs Epochs – single trial (Part 1d)(a)

On average, it seemed to reach a total error of less than 0.05 around ~20000 epochs (which is longer). The descent looks like it is less steep than before (less of an S-shape).

2.4.2 b

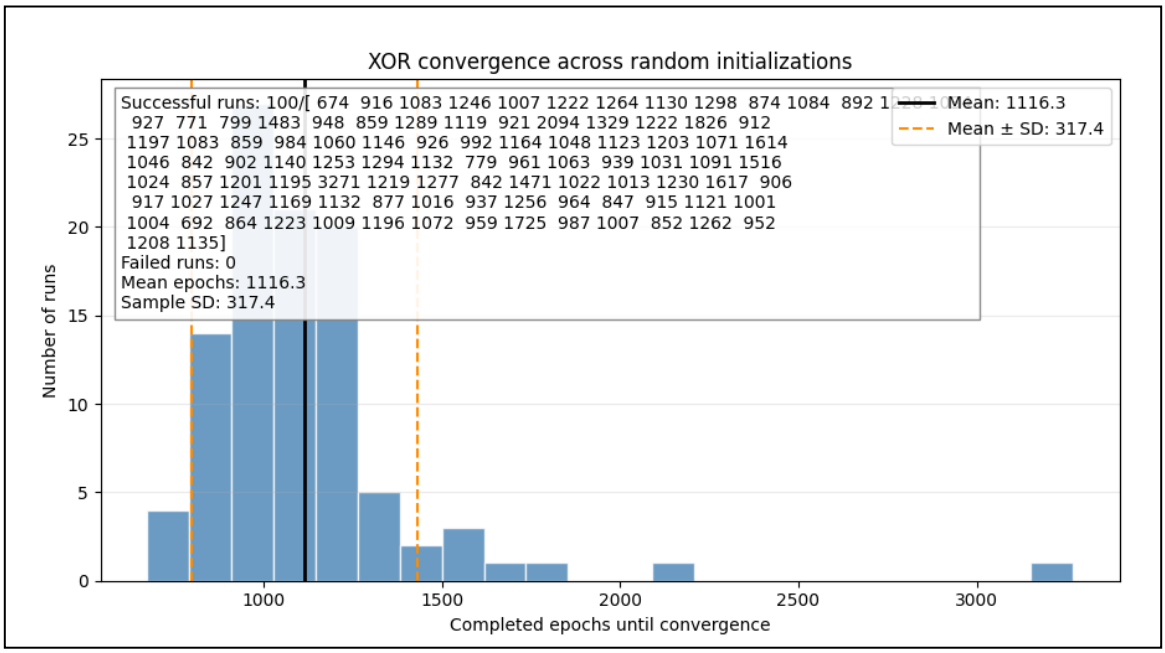


Figure 11 – Error vs Epochs over 100 trials – Bipolar (Part 1d)(b)

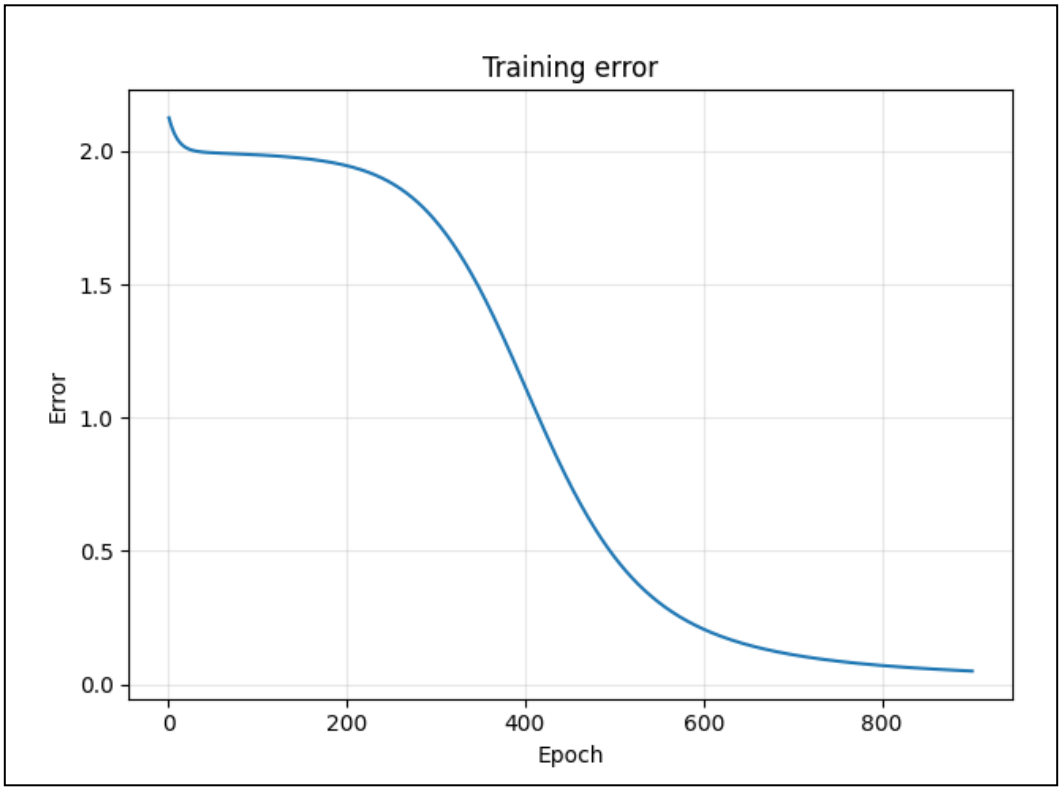


Figure 12 – Error vs Epochs – single trial (Part 1d)(b)

On average, it seemed to reach a total error of less than 0.05 around ~1000 epochs (also longer than before).

2.4.3 c

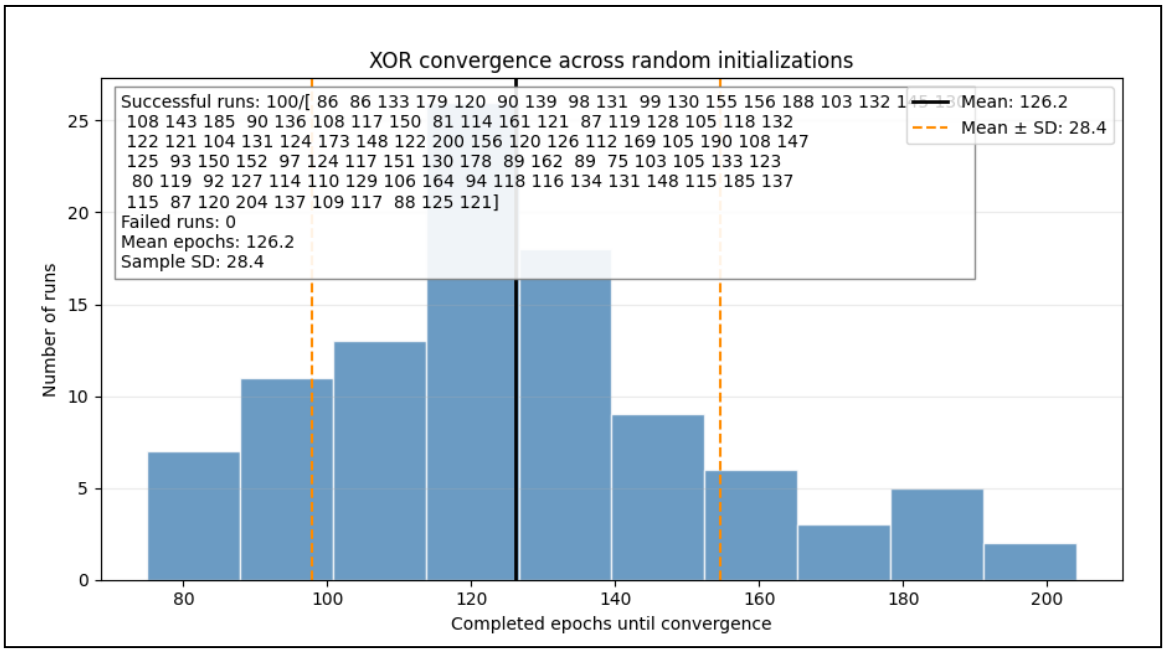


Figure 13 – Error vs Epochs over 100 trials – Bipolar w/ Momentum (Part 1d)(c)

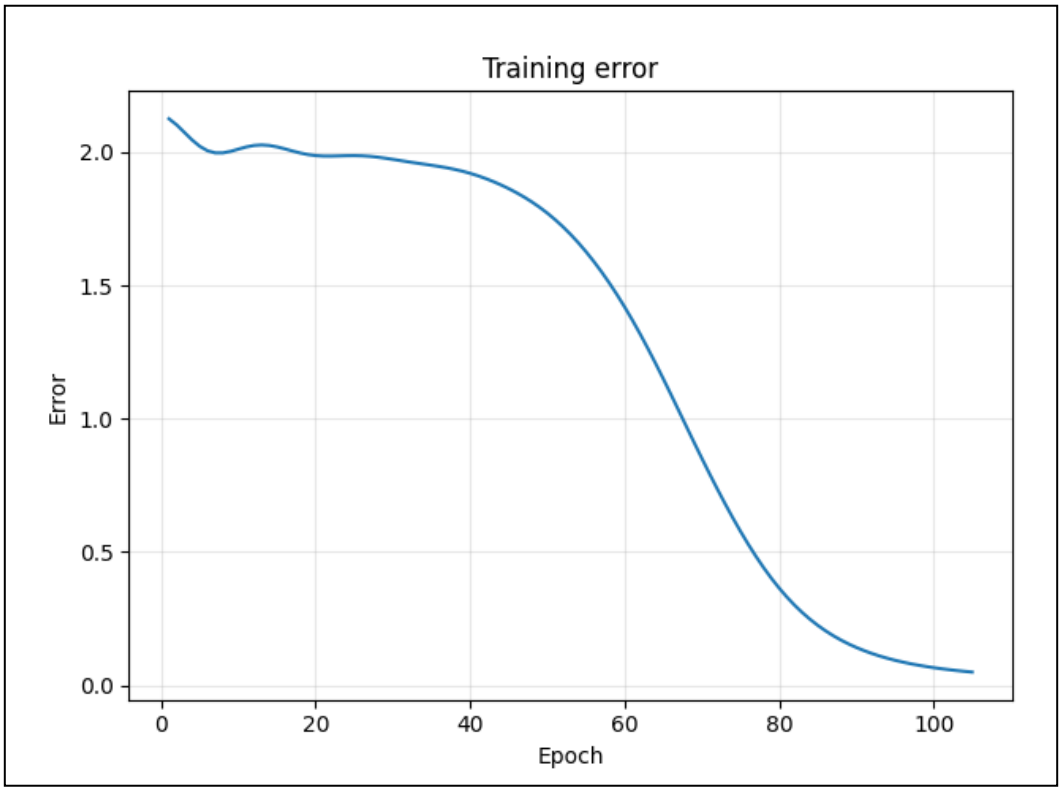


Figure 14 – Error vs Epochs – single trial (Part 1d)(c)

On average, it seemed to reach a total error of less than 0.05 around ~120 epochs (also longer than before). This is almost double what online mode would take, however it is still less than online mode without momentum. The curves are still smooth, but the performance isn't as good. Perhaps this could be because we aren't running backpropagation weight updates 4 times per epoch like we were with online training.

3 Appendix

3.1 NNInterface

```
from BaseInterface import BaseInterface, ACTIV, LEARN
import numpy as np
import math

class NNInterface(BaseInterface):
    def __init__(self):
        super().__init__()
        self._errorThresh: float = 0.05


        self.inputs:     np.ndarray
        self.nodes:     np.ndarray
        self.weights:   list[np.ndarray]
        self.weightedSum: list[float]

        self.outputs: np.ndarray

    def initWeights(self, initialWeightRange) -> None:
        pass

    def zeroWeights(self):
        pass

class NeuralNetwork(NNInterface):
    def __init__(self, inputs: int = 2, hiddenNodes: list[int] | None = [4, 4], outputs: int = 1,
                 learningRate: float = 0.2, momentum: float = 0.0,
                 activFunc: ACTIV = ACTIV.kBinary, learnMode: LEARN = LEARN.kOnline, initialWeightRange:
tuple[float, float] = (-0.5, 0.5), maxEpoch: int = 10000):
        super().__init__()

        if inputs < 2 or outputs < 1 or hiddenNodes is None or len(hiddenNodes) < 1:
            raise ValueError(f"inputs ({inputs}), outputs ({outputs}), and hiddenNodes ({hiddenNodes})
must be positive")

        self.learningRate = learningRate
        self.momentum = momentum
        self.activation = NNInterface.Activation(activation=activFunc)
        self.learningMode = learnMode
        self.rng = np.random.default_rng(4)
        self._errorThresh = 0.05
        self._maxEpoch = maxEpoch

        self._layerSizes = [inputs, *hiddenNodes, outputs]
        self._outputs = np.array([])
        self._lastWeights: list[np.ndarray]

        self.inputs = np.zeros(inputs)
        self.nodes = [np.zeros(n) for n in hiddenNodes]
        self.output = np.zeros(outputs)

        # Example: [2, 4, 4, 1] means input -> hidden -> hidden -> output.

        self.weightedSum: list
        self.initialWeightRange = initialWeightRange
        self.initWeights(initialWeightRange)

    def initWeights(self, initialWeightRange: tuple[float, float]) -> None:
        low, high = initialWeightRange
        if low >= high:
            raise ValueError("initialWeightRange must have low < high")

        self.weights = [
            self.rng.uniform(low, high, size=(n_in + 1, n_out))
            for n_in, n_out in zip(self._layerSizes[:-1], self._layerSizes[1:])
        ]
        self._lastWeights = [w.copy() for w in self.weights]

    def zeroWeights(self) -> None:
        for w in self.weights:
            w.fill(0.0)
```

```
        for update in self._lastWeights:
            update.fill(0.0)

    def fwdProp(self) -> float:
        output: float = 0.0
        self.weightedSums = []

        for index, layer in enumerate(self.weights):
            if index == len(self.weights) - 1:
                self.weightedSums.append(np.matmul(layer.T, np.append(self.nodes[index - 1], 1)))
                output: float = self.activation.function(self.weightedSums[-1])
                break

            if index == 0:
                self.weightedSums.append(np.matmul(layer.T, np.append(self.inputs, 1)))
                self.nodes[index] = self.activation.function(self.weightedSums[-1])
            else:
                self.weightedSums.append(np.matmul(layer.T, (np.append(self.nodes[index - 1], 1))))
                self.nodes[index] = self.activation.function(self.weightedSums[-1])

        return output
        # if len(self._outputs) > 1:
        #     self._outputs = np.append(self._outputs, output)
        # else:
        #     self._outputs = np.array([self.output])

    def halfTotalSquareError(self):
        error = np.subtract(self._outputs.ravel(), self.expectedOutputs.ravel()).T # vector subtraction
        return 0.5 * np.dot(error,error)

    def backProp(self, target: float) -> list[np.ndarray]:
        # deltas[i] belongs to the neurons reached by self.weights[i].
        # there should be deltas for each node + the output node
        deltas = [None] * len(self.weights)

        # Output layer: (target - output) * activation derivative
        deltas[-1] = ((target - self._outputs[-1]) * self.activation.derivative(self.weightedSums[-1]))

        # Hidden layers in reverse order.
        for layer in range(len(self.weights) - 2, -1, -1):
            # get delta from layer above
            delta = deltas[layer + 1]

            # delete the bias row from weights, and then dot product between weights and delta
            w = np.delete(self.weights[layer + 1], -1, axis=0) #self.weights[layer + 1][:-1, :]
            weighted_error = w @ delta

            # calculate the delta for the next row
            deltas[layer] = weighted_error * self.activation.derivative(self.weightedSums[layer])

        # Now that every delta uses the original weights, update the weights.
        changes : list[np.ndarray] = []
        for layer, weights in (enumerate(self.weights)):
            # add bias row back into inputs
            previous_input = np.append(self.inputs if layer == 0 else self.nodes[layer-1], 1.0)

            # use outer product (mat mul) to get weight matrix size correct and scalar mult by learning
rate
            changes.append(self.learningRate * np.outer(previous_input, deltas[layer]))

        # return changes to be applied later (depending on online or batch mode)
        return changes

    def backPropSequential(self, target: float) -> None:
        # Reconstruct the current output from the saved forward-pass sum.
        output = self.activation.function(self.weightedSums[-1])
        target = np.asarray(target, dtype=float).reshape(-1)

        delta = (target - output)* self.activation.derivative(self.weightedSums[-1])

        # Update the output layer first, then move toward the input.
        for layer in range(len(self.weights) - 1, -1, -1):
            if layer < len(self.weights) - 1:
                # The next layer's weights have ALREADY been updated.
                next_weights = self.weights[layer + 1][:-1, :]
```

#1

```
                delta = (next_weights @ delta) * self.activation.derivative(self.weightedSums[layer])

            previous_output = self.inputs if layer == 0 else self.nodes[layer - 1]

            previous_with_bias = np.append(previous_output, 1.0)

            # Includes the bias-weight update in the final row.
            change = self.learningRate * np.outer(previous_with_bias, delta)
            self.updateWeights(layer, change)

    def updateWeights(self, layer: int, change: np.ndarray) -> None:
        current = self.weights[layer]
        previous_update = current - self._lastWeights[layer]

        update = change + self.momentum * previous_update

        # Save the pre-update weights. A copy prevents shared-array problems.
        self._lastWeights[layer] = current.copy()
        current += update

    def trainModel(self, sequentialUpdates: bool = False, plot: bool = True, interactive: bool = False)
-> tuple[bool, list[float]]:
        self.loadTrainingSet()
        error: list[float] = []
        success = False

        if sequentialUpdates and self.learningMode != LEARN.kOnline:
            raise ValueError("Sequential layer updates require online mode")

        for epoch in range(1, self._maxEpoch + 1):
            self.expectedOutputs = np.array([])
            self._outputs = np.array([])
            avg_change = []
            for ip,op in self.trainingSet["XOR"].items():
                # set inputs and expected outputs
                self.inputs = np.array(ip) # set inputs
                self.expectedOutputs = np.append(self.expectedOutputs, op)

                # do forward propagation once for input set and compute error
                self._outputs = np.append(self._outputs, self.fwdProp())
                d = self.halfTotalSquareError()

                # apply back propagation to online learning mode
                if sequentialUpdates:
                    self.backPropSequential(target=op)
                else:
                    # do back propagation step
                    changes = self.backProp(target=op)
                    if self.learningMode == LEARN.kOnline:
                        for layer, change in enumerate(changes):
                            self.updateWeights(layer, change)
                    else:
                        avg_change.append(changes)
                        # self.weights = [w + dw for w, dw in zip(self.weights, avg_change.pop())]

            if self.learningMode == LEARN.kBatch:
                for layer, layer_changes in enumerate(zip(*avg_change)):
                    change = np.mean(layer_changes, axis=0)
                    self.updateWeights(layer, change)

            # do forward propagation again
            predictions = []
            self.expectedOutputs = np.array([])
            for ip,op in self.trainingSet["XOR"].items():
                # set inputs and expected outputs
                self.inputs = np.array(ip)  # set inputs
                self.expectedOutputs = np.append(self.expectedOutputs, op)
                predictions.append(self.fwdProp())

            # compute error
            self._outputs = np.asarray(predictions)
            error.append(self.halfTotalSquareError())

            if error[-1] <= self._errorThresh:
                success = True
```

```
                break

        if interactive:
            print(f"Reached {epoch} training iterations and achieved {error[-1]} error")
        if plot:
            self.plotGraph(error)
            self.plotNetwork()
            self.showPlots()
        if interactive:
            option = input("Save weights? [Y/n]")
            if option == "Y" or option == "y":
                filename = input("Enter file name: ")
                self.saveWeights(filename)
                print(f"{filename}.nnw saved, exiting...")
            elif option == "N" or option == "n":
                print("Training finished, exiting...")
            else:
                print("Invalid option, exiting...")

        return (success, error)

    def avgTrain(self, trials: int = 100, plot: bool = True, seed: int | None = None):
        if trials < 1:
            raise ValueError("trials must be at least 1")

        # Optional seed makes the whole experiment repeatable.
        experiment_rng = np.random.default_rng(seed)
        epochs = []
        trial_seeds = []

        for i in range(trials):
            trial_seed = int(experiment_rng.integers(0, 2 ** 32))
            trial_seeds.append(trial_seed)

            # Start a fresh model, including zeroed momentum history.
            self.rng = np.random.default_rng(trial_seed)
            self.initWeights(self.initialWeightRange)
            seq = (self.learningMode == LEARN.kOnline)
            success, loss = self.trainModel(sequentialUpdates = seq, plot = False, interactive = False)
            print(f"Trained model {i}")
            if success:
                epochs.append(len(loss))


        epochs = np.asarray(epochs, dtype=int)
        successes = len(epochs)
        failures = trials - successes

        mean = float(np.mean(epochs)) if successes else float("nan")
        sd = float(np.std(epochs, ddof=1)) if successes > 1 else float("nan")

        if plot:
            self.plotStatistics(epochs, successes, failures, epochs, mean, sd)

        return {
            "epochs": epochs,
            "mean": mean,
            "sd": sd,
            "successes": successes,
            "failures": failures,
            "seeds": trial_seeds,
        }
```

Figure 15 – NNInterface.py

3.2 BaseInterface

```
from abc import ABC, abstractmethod
from enum import Enum
from dataclasses import dataclass
from fileinput import filename
```

#1

```
import matplotlib.pyplot as plt
import numpy as np
import yaml

class NET(Enum):
    IP = "Input Layer"
    HD = "Hidden Layer"
    OP = "Output Layer"
    WT = "Weight"

class ACTIV(Enum):
    kBinary     = "Binary"
    kBipolar    = "Bipolar"

class LEARN(Enum):
    kBatch     = "Batch"
    kOnline    = "Online"

class BaseInterface(ABC):
    class Activation():
        def __init__(self, activation: ACTIV = ACTIV.kBinary):
            self.activation = activation

        def function(self, x: np.ndarray) -> float:
            if self.activation == ACTIV.kBinary:
                return self.binary_sigmoid(x)
            elif self.activation == ACTIV.kBipolar:
                return self.bipolar_sigmoid(x)
            else:
                raise ValueError(f"Unknown activation function: {self.activation}")

        def derivative(self, x) -> float:
            if self.activation == ACTIV.kBinary:
                return self.deriv_binary_sigmoid(x)
            elif self.activation == ACTIV.kBipolar:
                return self.deriv_bipolar_sigmoid(x)
            else:
                raise ValueError(f"Unknown activation function: {self.activation}")

        def bipolar_sigmoid(self, x: np.ndarray | float) -> np.ndarray | float:
            return 2 / (1 + np.exp(-x)) - 1

        def deriv_bipolar_sigmoid(self, x: np.ndarray | float) -> np.ndarray | float:
            return 0.5 * (1 - (self.bipolar_sigmoid(x) ** 2))

        def binary_sigmoid(self, x: np.ndarray | float) -> np.ndarray | float:
            return 1 / (1 + np.exp(-x))

        def deriv_binary_sigmoid(self, x: np.ndarray | float) -> np.ndarray | float:
            return self.binary_sigmoid(x) * (1 - self.binary_sigmoid(x))


    @abstractmethod
    def __init__(self):
        self.weights: list[np.ndarray]

        self._layerSizes: list[int]
        self._outputs: np.ndarray

        self.inputs: np.ndarray
        self.nodes: list[np.ndarray]
        self.output: np.ndarray

        self.activation: BaseInterface.Activation
```

#1

```
def loadTrainingSet(self) -> None:
    if self.activation.activation == ACTIV.kBinary:
        filename = "BinaryTrainingSet.yaml"
    elif self.activation.activation == ACTIV.kBipolar:
        filename = "BipolarTrainingSet.yaml"
    else:
        raise ValueError(f"Unknown activation function: {self.activation.activation}")

    with open(filename, encoding="utf-8") as file:
        data = yaml.safe_load(file)

    self.trainingSet = {}

    for gate_name, rows in data.items():
        self.trainingSet[gate_name] = {
            (row["inputs"]["x1"], row["inputs"]["x2"]): row["outputs"]["y"]
            for row in rows.values()
        }

def saveWeights(self, filename: str = "weights.npz") -> None:
    data = {
        "layerSizes": np.asarray(self._layerSizes, dtype=int),
        **{
            f"weight_{i}": weights
            for i, weights in enumerate(self.weights)
        },
    }

    # Opening the file explicitly preserves the exact filename.
    with open("weights/"+filename+".nnw", "wb") as file:
        np.savez_compressed(file, **data)

def loadWeights(self, filename: str = "weights.npz") -> None:
    with np.load(filename, allow_pickle=False) as data:
        saved_sizes = data["layerSizes"].tolist()

        if saved_sizes != self._layerSizes:
            raise ValueError(
                f"Saved architecture {saved_sizes} does not match "
                f"this network's architecture {self._layerSizes}"
            )

        loaded_weights = []

        for i, (n_in, n_out) in enumerate(
                zip(self._layerSizes[:-1], self._layerSizes[1:])
        ):
            weights = np.array(data[f"weight_{i}"], dtype=float, copy=True)
            expected_shape = (n_in + 1, n_out)

            if weights.shape != expected_shape:
                raise ValueError(
                    f"Weight {i} has shape {weights.shape}; "
                    f"expected {expected_shape}"
                )

            if not np.all(np.isfinite(weights)):
                raise ValueError(f"Weight {i} contains invalid values")

            loaded_weights.append(weights)

    # Replace existing weights only after all matrices pass validation.
    self.weights = loaded_weights
```

#1

```
        self.previousUpdates = [np.zeros_like(w) for w in self.weights]

    def plotGraph(self, error_history: list[float]):
        # error_history: list[float], one value per completed epoch
        fig, ax = plt.subplots()
        plt.plot(range(1, len(error_history) + 1), error_history)
        plt.xlabel("Epoch")
        plt.ylabel("Error")
        plt.title("Training error")
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        # plt.show()

    def plotNetwork( # Thank you chatGPT
            self,
            showWeights: bool = True,
            decimals: int = 2,
    ):
        sizes = self._layerSizes
        max_nodes = max(sizes)
        bias_y = -(max_nodes - 1) / 2 - 1.5

        fig, ax = plt.subplots(
            figsize=(max(8, 3 * len(sizes)), max(5, max_nodes + 2))
        )

        # Position each layer vertically around y = 0.
        positions = [
            [(layer, (count - 1) / 2 - neuron) for neuron in range(count)]
            for layer, count in enumerate(sizes)
        ]

        max_weight = max(
            float(np.max(np.abs(w))) for w in self.weights
        )
        max_weight = max(max_weight, 1e-12)

        # Connections, including the final bias row of each matrix.
        for layer, weights in enumerate(self.weights):
            sources = positions[layer] + [(layer, bias_y)]
            destinations = positions[layer + 1]

            for i, (x1, y1) in enumerate(sources):
                for j, (x2, y2) in enumerate(destinations):
                    weight = float(weights[i, j])
                    color = (
                        "tab:blue" if weight > 0
                        else "tab:red" if weight < 0
                        else "gray"
                    )

                    ax.plot(
                        [x1, x2], [y1, y2],
                        color=color,
                        linewidth=0.5 + 2 * abs(weight) / max_weight,
                        alpha=0.6,
                        zorder=1,
                    )

                    if showWeights:
                        # Stagger labels along the connections.
                        t = 0.35 + 0.35 * i / max(1, len(sources) - 1)
                        ax.text(
                            x1 + t * (x2 - x1),
```

```
                    y1 + t * (y2 - y1),
                    f"{weight:.{decimals}f}",
                    fontsize=7,
                    ha="center",
                    va="center",
                    color=color,
                    bbox=dict(
                        facecolor="white", edgecolor="none",
                        alpha=0.85, pad=0.5,
                    ),
                    zorder=2,
                )

    # Neurons.
    for layer, layer_positions in enumerate(positions):
        for neuron, (x, y) in enumerate(layer_positions):
            label = (
                f"x{neuron + 1}" if layer == 0
                else f"y{neuron + 1}" if layer == len(sizes) - 1
                else f"h{layer},{neuron + 1}"
            )
            ax.scatter(
                x, y, s=900, facecolor="white",
                edgecolor="black", zorder=3,
            )
            ax.text(x, y, label, ha="center", va="center", zorder=4)

        if layer < len(sizes) - 1:
            ax.scatter(
                layer, bias_y, s=700, facecolor="lightgray",
                edgecolor="black", zorder=3,
            )
            ax.text(
                layer, bias_y, "1",
                ha="center", va="center", zorder=4,
            )

        heading = (
            "Input" if layer == 0
            else "Output" if layer == len(sizes) - 1
            else f"Hidden {layer}"
        )
        ax.text(
            layer, (max_nodes - 1) / 2 + 1,
            heading, ha="center", fontweight="bold",
        )

    ax.set_xlim(-0.4, len(sizes) - 0.6)
    ax.set_ylim(bias_y - 0.7, (max_nodes - 1) / 2 + 1.5)
    ax.set_title("Neural network — blue: positive, red: negative")
    ax.axis("off")
    fig.tight_layout()
    #plt.show()

    return fig, ax

def plotStatistics(self, trials, successes, failures, epochs, mean, sd,):

    plt.close("all")
    fig, ax = plt.subplots(figsize=(9, 5))

    if successes:
        ax.hist(
            epochs,
```

```
            bins="auto",
            color="steelblue",
            edgecolor="white",
            alpha=0.8,
        )

        ax.axvline(
            mean, color="black", linewidth=2,
            label=f"Mean: {mean:.1f}",
        )

        if successes > 1:
            for position in (mean - sd, mean + sd):
                ax.axvline(
                    position, color="darkorange",
                    linestyle="--", linewidth=1.5,
                )
            # Legend entry for the two dashed lines.
            ax.plot(
                [], [], color="darkorange", linestyle="--",
                label=f"Mean ± SD: {sd:.1f}",
            )

        ax.legend(loc="upper right")

    mean_text = f"{mean:.1f}" if successes else "N/A"
    sd_text = f"{sd:.1f}" if successes > 1 else "N/A"

    ax.text(
        0.02, 0.97,
        f"Successful runs: {successes}/{trials}\n"
        f"Failed runs: {failures}\n"
        f"Mean epochs: {mean_text}\n"
        f"Sample SD: {sd_text}",
        transform=ax.transAxes,
        va="top",
        bbox=dict(facecolor="white", edgecolor="gray", alpha=0.9),
    )

    ax.set(
        xlabel="Completed epochs until convergence",
        ylabel="Number of runs",
        title="XOR convergence across random initializations",
    )
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    plt.show()

def showPlots(self):
    plt.show()
# @abstractmethod
# def loadWeights(self): pass

# @abstractmethod
# def saveWeights(self): pass

# @abstractmethod
# def trainModel(self): pass
```

Figure 16 – BaseInterface.py

#1

3.3 LUTInterface, main.py, and Training Sets

```
from BaseInterface import BaseInterface, NET

class LUTInterface(BaseInterface):
    def __init__(self):
        super().__init__()
```

Figure 17 – LUTInterface.py

```
from NNInterface import NeuralNetwork
from BaseInterface import ACTIV, LEARN
from LUTInterface import LUTInterface

# Binary Activation function for 2-4-1 network
myNet = NeuralNetwork(hiddenNodes=[4], maxEpoch=50000, activFunc=ACTIV.kBipolar, learnMode=LEARN.kBatch, momentum=0.9)
myNet.trainModel()
# myNet.avgTrain(100, plot=True)
```

Figure 18 – main.py

```
XOR:
  1:
    inputs:   {x1: 0, x2: 0}
    outputs:  {y: 0}
  2:
    inputs:   {x1: 0, x2: 1}
    outputs:  {y: 1}
  3:
    inputs:   {x1: 1, x2: 0}
    outputs:  {y: 1}
  4:
    inputs:   {x1: 1, x2: 1}
    outputs:  {y: 0}
```

Figure 19 – BinaryTrainingSet.yaml

```
XOR:
  1:
    inputs:   {x1: -1, x2: -1}
    outputs:  {y: -1}
  2:
    inputs:   {x1: -1, x2: 1}
    outputs:  {y: 1}
  3:
    inputs:   {x1: 1, x2: -1}
    outputs:  {y: 1}
  4:
    inputs:   {x1: 1, x2: 1}
    outputs:  {y: -1}
```

Figure 20 – BipolarTrainingSet.yaml